

Mini Project 1

Path 3 Data Security in Model Training

Name: Gabriel Bazalar

Purdue User: gbazalar

<https://github.com/ECEDataScience/miniproject-f23-gbazalar.git>

Summary

In this report we tested the robustness of three main models to analyze data sets. The MLP, K-Neighbor, and Gaussian model are extensively analyzed through different tests. We also explain two methods for modifying the data set: poison with random numbers and denoising with KernelPCA.

Dataset Description

To start this project, it's important to first understand the sklearn dataset we'll be using. This dataset contains 1797 samples, each showing a different handwritten digit displayed in an 8 x 8 matrix. According to the sklearn documentation, each point in the matrix represents a pixel with an intensity level that ranges from 0 to 16. There are ten classes overall, corresponding to digits 0 through 9. This is a key part of our project, as we'll be inputting these 8 x 8 matrices into various models and then assessing how accurately each model can predict the correct label, which is the actual number that each image represents.

Description of the Models Used

Gaussian Naive Bayes

This model combines Bayes' theorem and the concept of Gaussian distribution to make predictions on datasets. This means the model can update its results with new knowledge it acquires. It also uses the principle of Gaussian distribution, where data is assumed to be normally distributed around the mean, to make assumptions about new data. In our code, the model is fed with a training dataset, starting by classifying each number and refining its assumptions with new iterations of the same number. For example, it will see a '2' represented as a 64-digit input (the 8 x 8 grid), label it, and improve its accuracy as it analyzes more '2's. It's vital to train this model with a variety of classes (different numbers) since it learns by distinguishing between their features. Its accuracy would be hardly diminished when the features of the data set are correlated.

K-Neighbors Classifier

This model operates on the principle of proximity, using the K-Neighbors voting system to make predictions on datasets. It works by identifying the 'k' nearest data points to a new sample and then classifying the sample based on the majority label among these neighbors. In our project, the model will be trained with a dataset of digit images. It will classify a digit by considering the closest examples in its training data, improving its accuracy as it encounters more variations of each digit. It's crucial for this model to be trained with diverse examples, as it relies on comparing new data points with known

samples. Its performance can be affected by imbalanced datasets where some classes dominate over others.

MLPClassifier

The MLPClassifier, or Multi-Layer Perceptron Classifier, is a type of neural network model for predictive analysis. It learns by passing data through multiple layers, each with its own set of neurons that process and transform the data. In our code, we use it to classify digit images from an 8x8 grid into their respective numbers. The model learns by adjusting weights and biases across its layers, refining its predictions with each iteration of the training data. That means that it will require a large amount of data to perform well. As this model is much more complex than the previous ones, it has different hyperparameters that can drastically change the output of the neural network.

Coding Process

Getting familiarized with plotting the data set:

To start the task at hand, we need to prepare our data. After importing the necessary libraries.

```
#MiniProjectPath3
import numpy as np
import matplotlib.pyplot as plt
# Import datasets, classifiers and performance metrics
from sklearn import datasets, svm, metrics
from sklearn.model_selection import train_test_split
#import models
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.neural_network import MLPClassifier
import matplotlib.pyplot as plt
import copy
from sklearn.decomposition import KernelPCA
```

We proceed by retrieving and splitting the essential information into different variables for training our models. In this case, we've separated the data into X_train, X_test, y_train, and y_test using the train_test_split function. This function allows us to obtain the datasets that we will use for training and testing our models. We've set our parameters so that 60% of our data will be used for testing, while the remaining 40% is designated for training purposes.

```
rng = np.random.RandomState(1)
digits = datasets.load_digits()
images = digits.images
labels = digits.target

#Get our training data
X_train, X_test, y_train, y_test = train_test_split(images, labels,
test_size=0.6, shuffle=False)
```

1.Search and print functions

Now that we've classified the dataset into their corresponding variables, we can create functions to print results and retrieve data after training each model. We'll have two functions: dataset_searcher and print_number. These will operate post-training for each model. The dataset_searcher is fed with a list of

numbers, a dataset of images, and their labels. It filters and retrieves specific number images for plotting, in my case, defined as the first instance of each number in the dataset. Then, it converts these images into NumPy arrays to be used in training models.

```
def dataset_searcher(number_list, images, labels):
    selected_images = []
    selected_labels = []

    for number in number_list:
        index = np.where(labels == number)[0][0]
        selected_images.append(images[index])
        selected_labels.append(labels[index])

    images_narray = np.array(selected_images)
    labels_narray = np.array(selected_labels)
    return images_narray, labels_narray
```

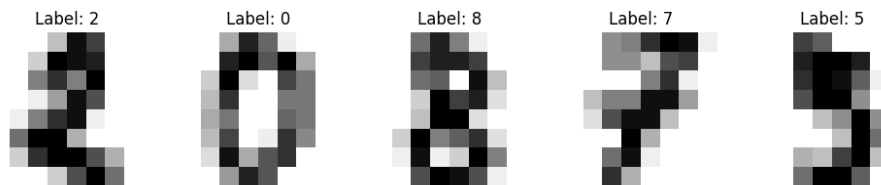
The `print_numbers` function will be useful to visually display a set of digit images along with their corresponding labels. We feed the function with arrays of images and labels, so it creates a series of subplots, each representing one image. For every digit in the given array, the function plots the image in grayscale and annotates it with its respective label. This visual representation is especially useful for inspecting the model's classification results, allowing us to see both the digit image and what the model has identified it as.

```
def print_numbers(images, labels):
    n_images = len(images)

    plt.figure(figsize=(10, 2))
    for i in range(n_images):
        plt.subplot(1, n_images, i + 1)
        plt.imshow(images[i], cmap=plt.cm.gray_r)
        plt.title(f'Label: {labels[i]}')
        plt.axis('off')
    plt.tight_layout()
    plt.show()
```

2. Plot first set of numbers

To test the functions we printed the set of numbers [2, 0, 8, 7, 5] with it respective labels.



```
#Part 2
print_numbers(class_number_images , class_number_labels )
```

3. Gaussian Naive Bayes model predictions

We will now use the Gaussian Naive Bayes model to predict number labels on the previously set numbers. Since these models don't accept 2D matrices (like the 8 x 8 matrix) as input, we need to reshape them. The model will be trained with 718 images, each consisting of 64 values. After training the model with this dataset, we'll also reshape our test data and use it to test the model's performance.

```
model_1 = GaussianNB()

#however, before we fit the model we need to change the 8x8 image data into 1
dimension
# so instead of having the Xtrain data beign of shape 718 (718 images) by 8 by 8
# the new shape would be 718 by 64
X_train_resaped = X_train.reshape(X_train.shape[0], -1)

#Now we can fit the model
model_1.fit(X_train_resaped, y_train)
#Part 3 Calculate model1_results using model_1.predict()
X_test_resaped = X_test.reshape(X_test.shape[0], -1)
model1_results = model_1.predict(X_test_resaped)
classnumbers_prediction =
model_1.predict(class_number_images.reshape(class_number_images.shape[0], -1))
print_numbers(class_number_images,classnumbers_prediction)
```

4. Accuracy function and Gaussian model result

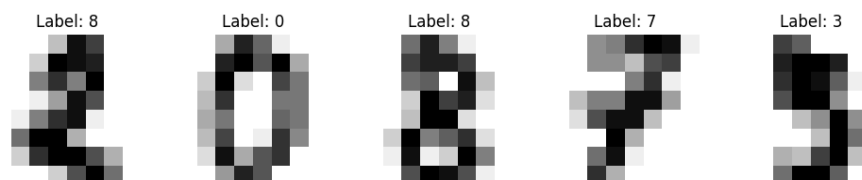
In order to get an standardized metric to compare all the models we employed in this path, we defined a function to get an accuracy percentage of each model. The function OverallAccuracy compares the model's predicted results with the actual values to calculate a percentage of how well it performed. Specifically, it counts how many predictions made by the model match the true labels (correct_predictions) and divides this number by the total number of predictions made (total_predictions). The result is the accuracy rate, a measure of how often the model correctly identifies the digits.

```
def OverallAccuracy(results, actual_values):
    correct_predictions = np.sum(results == actual_values)
    total_predictions = len(results)
    Accuracy = correct_predictions / total_predictions
    return Accuracy
```

We employed this function to see how well the Gaussian Naive Bayes model performed. For this case we got the following print:

The overall results of the Gaussian model is 0.8007414272474513

This score says that it got approximately 80% of accuracy on predicting the labels of the image dataset.

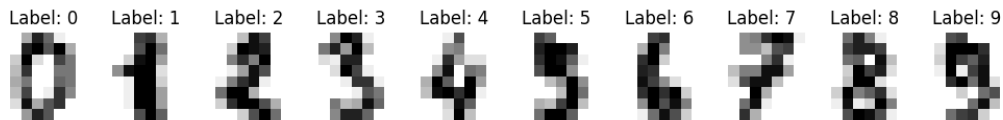


Although the accuracy tells us that the printed image should match 4/5 numbers, it did not. This discrepancy could be explained due to the fact that the actual accuracy test is runs a bigger data set and the plotted image is just an approximation of how the model behaves with those specific images of numbers.

5. Plot second set of numbers

Now we want to increase the number sample to test the other models (MLP and K-Neighbor). We will use all the 10 classes of the data set: [0,1,2,3,4,5,6,7,8,9]. In order to do this we have to call again the `dataset_searcher` function and we will test it by running the `print_numbers` function with its corresponding labels.

```
#Part 5
allnumbers = [0,1,2,3,4,5,6,7,8,9]
allnumbers_images, allnumbers_labels = dataset_searcher(allnumbers, images,
labels)
print_numbers(allnumbers_images, allnumbers_labels)
```



As we expected, all the classes came out correctly with their corresponding labels.

6. MLP and K-Neighbor models accuracy test with all classes

As we explained previously, the K-Neighbor model works by getting the distance to the nearest k data point. In this case, we decided to take 10 n neighbors into consideration since it seem to give the best result in the accuracy test and is reasonable amount of neighbors for the size of the data set.

```
model_2 = KNeighborsClassifier(n_neighbors=10)
model_2.fit(X_train_reshaped, y_train)
model2_results = model_2.predict(X_test_reshaped)
Model2_Overall_Accuracy = OverallAccuracy(model2_results, y_test)
print("The overall results of the K Nearest Neighbors model is " +
str(Model2_Overall_Accuracy))

allnumbers_predictions_model2 =
model_2.predict(allnumbers_images.reshape(allnumbers_images.shape[0], -1))
print_numbers(allnumbers_images, allnumbers_predictions_model2)
```

After initializing the model, we fed it with the reshaped (8 x 8 to 64) train data. Then we run the model to predict the new test data set. After calling the `Overall_Accuracy` function, we also called the `print_numbers` function and run the model with the [0, 1, 2, 3, 4, 5, 6, 7, 8, 9] images. We did the same process with the next model: MLP. This neural network works by initializing its weights as random values so that is why we input a fixed random seed for the model. The seed would ensure that we get a similar result every time we run the code.

```

model_3 = MLPClassifier(random_state=0)
model_3.fit(X_train_reshaped, y_train)
model3_results = model_3.predict(X_test_reshaped)
Model3_Overall_Accuracy = OverallAccuracy(model3_results, y_test)
print("The overall results of the MLP Classifier model is " +
      str(Model3_Overall_Accuracy))

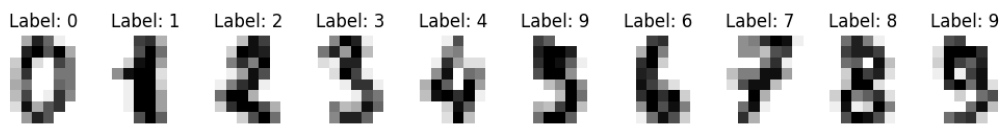
allnumbers_predictions_model3 =
model_3.predict(allnumbers_images.reshape(allnumbers_images.shape[0], -1))
print_numbers(allnumbers_images, allnumbers_predictions_model3)

```

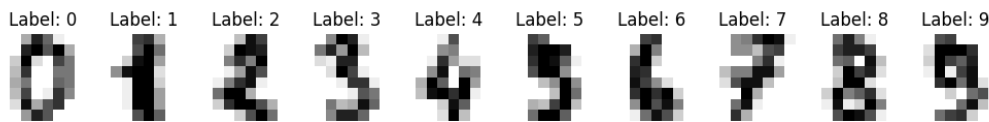
7. MLP and K-Neighbor results

The results on the models were:

The overall results of the K Nearest Neighbors model is 0.9545875810936052



The overall results of the MLP Classifier model is 0.9147358665430955



As we can observe, the model with the highest accuracy is the K nearest neighbor with approximately 95.5% of accuracy. The MLP model had reasonably high accuracy also. Since it is a more robust model, it requires a larger data set to overcome the K neighbor model. Moreover, we can also explain the higher accuracy of the K neighbor model with the fine tuning we performed on it. We specifically used 10 neighbors since it got the best accuracy. For the MLP model, we did not perform any fine tuning. In conclusion, between the three of them (MLP, K-Neighbor, and Gaussian) the most accurate model was the K-Neighbor.

8. "Poisoning" the data

To make our models more robust, we may train them with a more realistic example of data; thus, we "poison" the original dataset. "Poison" in this context means to add a bit of random variance to our original dataset. This requires generating a random value of noise with the rng function. The parameters inside the function ensure that we can control this noise and that it has the same shape as the original dataset, so we can later add it just by using simple addition in the code. The noise_scale variable is set to 10; the units of this number are related to how far from the standard deviation of the Gaussian distribution the noise will be generated.

```

#Part 8
#Poisoning
# Code for generating poison data. There is nothing to change here.
noise_scale = 10.0
poison = rng.normal(scale=noise_scale, size=X_train.shape)

X_train_poison = X_train + poison
reduced_X_train_poison = X_train_poison[allnumbers]

```

9. Feeding models with “poisoned” data

After poisoning the data set, we have to reshape it and then feed it again as we previously did with each model:

```

#Part 9-11
#Determine the 3 models performance but with the poisoned training data X_train_poison and
y_train instead of X_train and y_train
X_train_poison_resaped = X_train_poison.reshape(X_train_poison.shape[0], -1)

model_1_poison = GaussianNB()
model_1_poison.fit(X_train_poison_resaped, y_train)
model1_poison_results = model_1_poison.predict(X_test_resaped)
Model1_poison_Overall_Accuracy = OverallAccuracy(model1_poison_results, y_test)
print("The overall results of the (Poisoned) Gaussian model is " +
str(Model1_poison_Overall_Accuracy))

model_2_poison = KNeighborsClassifier(n_neighbors=10)
model_2_poison.fit(X_train_poison_resaped, y_train)
model2_poison_results = model_2_poison.predict(X_test_resaped)
Model2_poison_Overall_Accuracy = OverallAccuracy(model2_poison_results, y_test)
print("The overall results of the (Poisoned) K Nearest Neighbors model is " +
str(Model2_poison_Overall_Accuracy))

model_3_poison = MLPClassifier(random_state=0)
model_3_poison.fit(X_train_poison_resaped, y_train)
model3_poison_results = model_3_poison.predict(X_test_resaped)
Model3_poison_Overall_Accuracy = OverallAccuracy(model3_poison_results, y_test)
print("The overall results of the (Poisoned) MLP Classifier model is " +
str(Model3_poison_Overall_Accuracy))

poison_allnumbers_images_resaped =
reduced_X_train_poison.reshape(reduced_X_train_poison.shape[0], -1)

poison_allnumbers_predictions_gnb =
model_1_poison.predict(poison_allnumbers_images_resaped)
print_numbers(reduced_X_train_poison, poison_allnumbers_predictions_gnb)

poison_allnumbers_predictions_knn =
model_2_poison.predict(poison_allnumbers_images_resaped)
print_numbers(reduced_X_train_poison, poison_allnumbers_predictions_knn)

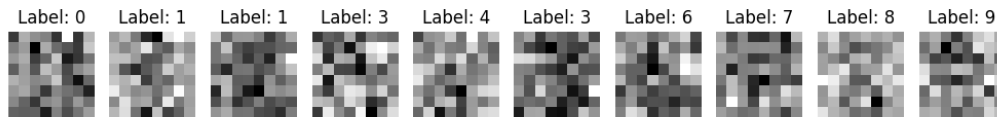
allnumbers_predictions_mlp = model_3_poison.predict(poison_allnumbers_images_resaped)
print_numbers(reduced_X_train_poison, allnumbers_predictions_mlp)

```

10. Poison data results

The results for each model were:

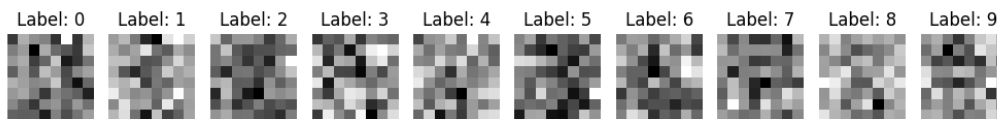
The overall results of the (Poisoned) Gaussian model is 0.8146431881371641



The overall results of the (Poisoned) K Nearest Neighbors model is 0.6051899907321594



The overall results of the (Poisoned) MLP Classifier model is 0.794253938832252



(Note that all the plots are tested with the poisoned data and not with the initial test data. The accuracy values are the important standardized measurement to explain the relation between the three models)

Let's try to explain the different results on accuracy for each model. As we mentioned before the numbers are not the best indicator of the accuracy since they are tested with the noised data and not with the original data for visual porpoises.

As we explained, the Gaussian model assumes that the features are normally distributed. By adding noise from a normal distribution, this assumption still holds, which explains why the model still performs relatively well even after the dataset is poisoned. The noise increases variance in the data, which the model can handle to a certain extent.

K-Neighbor model is very sensitive to noise because it relies on the distances between samples to make predictions. By adding noise, the distances between points in feature space are distorted, which leads to misclassification. A high noise_scale value like 10.0 can significantly disrupt the local neighborhoods that the model depends on, hence the substantial drop in accuracy.

Finally, the MLP model is a more robust to noise compared to K-Neighbor due to its ability to learn an abstract representation of the data through its layers. However, too much noise can still degrade its performance as it may start to fit the noise instead of the pattern.

11.Stronges model against poison

It is clear that for this scenario the Gaussian model is the most accurate, however this could dramatically change if we change the way in which we poison the data set. Right behind, the MLP model gets close to the accuracy of the Gaussian model since it is a more complex model and even after some fine tuning we could match the results or even improve them.

12.Denoising the data set with KernelPCA

Now, let's change our training dataset again by cleaning the poisoned dataset. We will do this by removing the noise from the data with a method found in the SKlearn library called Kernel PCA. This method for denoising data works by using Principal Component Analysis plus some kernel methods. In simple terms, PCA helps us find the main patterns in our data, and kernel methods assist us when these patterns are too complicated for PCA to handle alone. They work together to help us select the main patterns of the data and leave out the outliers that would be considered noise. This method have different parameters in its code that can help us fine tune the denoise functionality.

```
#Part 12-13
# Denoise the poisoned training data, X_train_poison.
# hint --> Suggest using KernelPCA method from sklearn library, for denoising
the data.
# When fitting the KernelPCA method, the input image of size 8x8 should be
reshaped into 1 dimension
# So instead of using the X_train_poison data of shape 718 (718 images) by 8 by
8, the new shape would be 718 by 64

X_train_poison_resaped = X_train_poison.reshape((X_train_poison.shape[0], -1))

kernel_pca = KernelPCA(n_components=None, kernel="rbf", gamma=1e-3,
fit_inverse_transform=True, alpha=5e-3,random_state=42,)

X_train_denoised_pca = kernel_pca.fit_transform(X_train_poison_resaped)

X_train_denoised = kernel_pca.inverse_transform(X_train_denoised_pca)

X_train_denoised = X_train_denoised.reshape(X_train_poison.shape)

reduced_X_train_denoised = X_train_denoised[allnumbers]
```

13. Difference between poison and denoised



Here we can observe the first three digits ([0,1,2]) of the original, poison, and denoise data set. As we can observe the poisoned data set is much less clear than the denoise data set. The KernelPCA did a great job of identifying just the close relationship between each number sample, leaving a much defined shape instead of different grayscale values all around.

14. Feeding models with "denoised" data

After denoising the data set, we have to reshape it and then feed it again as we previously did with each model:

```
#Part 14-15
#Determine the 3 models performance but with the denoised training data,
X_train_denoised and y_train instead of X_train_poison and y_train
#Explain how the model performances changed after the denoising process.

X_train_denoised_resaped = X_train_denoised.reshape(X_train_denoised.shape[0],
-1)

model_1_denoised = GaussianNB()
model_1_denoised.fit(X_train_denoised_resaped, y_train)
model1_denoised_results = model_1_denoised.predict(X_test_resaped)
Model1_denoised_Overall_Accuracy = OverallAccuracy(model1_denoised_results,
y_test)
print("The overall results of the (Denoise) Gaussian model is " +
str(Model1_denoised_Overall_Accuracy))

model_2_denoised = KNeighborsClassifier(n_neighbors=10)
model_2_denoised.fit(X_train_denoised_resaped, y_train)
model2_denoised_results = model_2_denoised.predict(X_test_resaped)
Model2_denoised_Overall_Accuracy = OverallAccuracy(model2_denoised_results,
y_test)
print("The overall results of the (Denoise) K Nearest Neighbors model is " +
str(Model2_denoised_Overall_Accuracy))

model_3_denoised = MLPClassifier(random_state=0, max_iter=500)
model_3_denoised.fit(X_train_denoised_resaped, y_train)
model3_denoised_results = model_3_denoised.predict(X_test_resaped)
Model3_denoised_Overall_Accuracy = OverallAccuracy(model3_denoised_results,
y_test)
print("The overall results of the (Denoise) MLP Classifier model is " +
str(Model3_denoised_Overall_Accuracy))

denoised_allnumbers_images_resaped =
reduced_X_train_denoised.reshape(reduced_X_train_denoised.shape[0], -1)

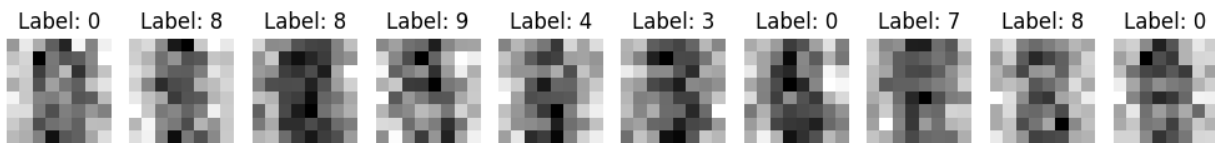
denoised_allnumbers_predictions_gnb =
model_1_poison.predict(denoised_allnumbers_images_resaped)
print_numbers(reduced_X_train_denoised, denoised_allnumbers_predictions_gnb)

denoised_allnumbers_predictions_knn =
model_2_poison.predict(denoised_allnumbers_images_resaped)
print_numbers(reduced_X_train_denoised, denoised_allnumbers_predictions_knn)
```

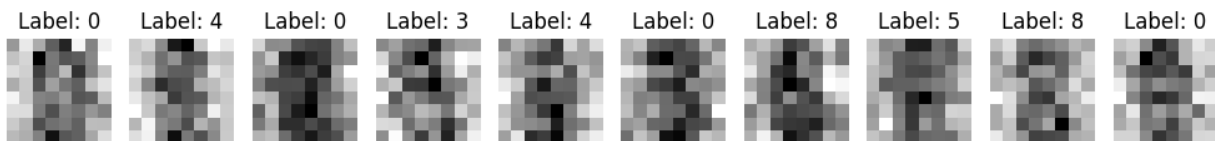
```
denoised_allnumbers_predictions_mlp =
model_3_poison.predict(denoised_allnumbers_images_reshaped)
print_numbers(reduced_X_train_denoised, denoised_allnumbers_predictions_mlp)
```

15. Denoised data results

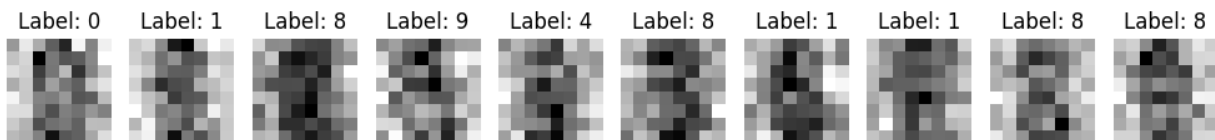
The overall results of the (Denoise) Gaussian model is 0.8489341983317887



The overall results of the (Denoise) K Nearest Neighbors model is 0.8266913809082483



The overall results of the (Denoise) MLP Classifier model is 0.830398517145505



(Note that all the plots are tested with the denoised data and not with the initial test data for visual purposes. The accuracy values are the important standardized measurement to explain the relation between the three models)

The Gaussian Naive Bayes model performed the best among the three with an accuracy of approximately 84.89%. As previously mentioned, the model assumes of normality in the data. The denoise process made the data more 'normally' distributed or at least more aligned with the assumptions the model, improving its performance.

The K-Neighbor model showed an accuracy of around 82.67%, which is a significant improvement from its performance on the poisoned data. The model is very sensitive to noise because it magnifies the distance

between data points. Denoising the data helped restoring the initial true distances between the data points, improving the model's accuracy.

The MLP model demonstrated an accuracy of approximately 83.04%, which is quite close to that of K-Neighbor. This model is capable of capturing complex patterns in data, but they can also be sensitive to the quality of the input data. The denoising process probably helped in clarifying the underlying patterns in the data, leading to improved performance.

The relatively lower performance compared to GNB might indicate that while MLP can handle complexity well, it might still be influenced by some residual noise or require further tuning (such as adjusting the number of layers, neurons, or learning rate) to achieve optimal performance.

In conclusion, the fact that all models performed better on the denoised data compared to the poisoned data demonstrates the effectiveness of KernelPCA in improving data quality for machine learning tasks.

Conclusion

After this extent analysis we can conclude that most robust model out of these three is the MLP model since it adapted better to all scenarios. Moreover, we also observed that the Gaussian model performs really well when the changes in data are related to the normal distribution. Those changes have a relatively small effect on the model performance. Finally we can say that the K-Neighbor model is the least adaptable model and should not be employed when there is going to be a large change in data sets since it is very susceptible to them.